# LAPORAN PRAKTIKUM
# PEMROGRAMAN BERORIENTASI OBJEK
# MODUL 4-5
# THE CORE OF OOP - ENCAPSULATION

**Disusun Oleh:**

**Adrenalin Syahrobby 105224030**

**PROGRAM STUDI ILMU KOMPUTER**

**FAKULTAS SAINS DAN KOMPUTER**

**UNIVERSITAS PERTAMINA**

**2026**

## I. Pendahuluan

Pada praktikum Pemrograman Berorientasi Objek Modul 4 dan 5 ini, dipelajari konsep inti dari Object-Oriented Programming (OOP) yang meliputi penggunaan Class, Object, Method, Constructor, serta prinsip Encapsulation. Pemahaman konsep ini sangat fundamental karena memungkinkan pengembang untuk membangun kode yang lebih terstruktur, modular, dan mudah dikelola dengan memetakan entitas dunia nyata ke dalam struktur program Java. Selain itu, mahasiswa juga dilatih untuk menerapkan Information Hiding guna melindungi integritas data dari akses langsung yang tidak sah melalui penggunaan access modifiers serta metode Getter dan Setter.

Pada tugas ini, dibuat sebuah program yang mensimulasikan sistem reservasi hotel untuk sebuah jaringan manajemen perhotelan guna mengatasi kendala operasional seperti overcapacity tamu dan penyalahgunaan kode voucher. Program ini mengimplementasikan kelas KamarHotel yang dilengkapi dengan berbagai atribut krusial seperti nomor kamar, tipe, kapasitas maksimal, harga, hingga status ketersediaan. Berdasarkan skenario yang diberikan, program melakukan proses validasi bisnis yang ketat, seperti standarisasi tipe kamar ("Reguler", "Premium", "Suite"), batas minimum harga sewa, hingga pengecekan kapasitas kamar saat proses pemesanan dilakukan.

Selain itu, program ini menerapkan konsep Method Overloading baik pada proses pemesanan kamar maupun pada sistem penagihan untuk memberikan fleksibilitas fungsionalitas berdasarkan input yang diberikan. Fitur perhitungan total bayar dirancang untuk mampu memproses penggunaan kode promo secara otomatis dengan syarat minimal durasi menginap tertentu. Melalui praktikum ini, diharapkan mahasiswa dapat memahami cara merancang arsitektur sistem yang aman dan kebal terhadap kesalahan operasional melalui integrasi prinsip enkapsulasi, validasi logika dalam setter, serta pemanfaatan konstruktor untuk inisialisasi objek secara efisien.

## II. Variabel

| No | Nama Variabel | Tipe data | Fungsi |
|---|---|---|---|
| 1 | kamar1 | KamarHotel | Objek untuk menyimpan data kamar hotel pertama (nomor 101, tipe Reguler, kapasitas 2) |
| 2 | kamar2 | KamarHotel | Objek untuk menyimpan data kamar hotel kedua (nomor 201, tipe Suite, kapasitas 2, harga 800000) |

| | | | |
|---|---|---|---|
| 3 | tagihanKamar 1 | double | Menyimpan hasil perhitungan total bayar untuk kamar 1 dengan voucher PROMO |
| 4 | tagihanKamar 2 | double | Menyimpan hasil perhitungan total bayar untuk kamar 2 dengan voucher PROMO |
| 6 | nomorKamar | String | Menyimpan nomor identitas kamar hotel |
| 7 | tipeKamar | String | Menyimpan tipe/jenis kamar (Reguler/Premium/Suite) |
| 8 | kapasitasMaksimal | int | Menyimpan jumlah maksimal tamu yang dapat menginap |
| 9 | hargaPerMalam | double | Menyimpan harga sewa kamar per malam |
| 10 | isTersedia | boolean | Menyimpan status ketersediaan kamar (true = tersedia, false = terisi) |
| 11 | totalTagihan | double | Variabel lokal untuk menyimpan hasil perhitungan sementara dalam method hitungTotalBayar |

## III. Constructor dan Method

| No | Nama Metode | Jenis Metode | Fungsi |
|---|---|---|---|
| 1 | KamarHotel (String nomorKamar, String tipeKamar, int kapasitasMaksimal) | constructor | Membuat objek kamar hotel dengan data dasar (nomor, tipe, kapasitas), harga default 0, status tersedia true |

| | | | |
|---|---|---|---|
| 2 | KamarHotel (String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) | constructor | Membuat objek kamar hotel lengkap dengan semua atribut termasuk harga per malam |
| 3 | getNomorKamar() | functional | Mengembalikan nilai nomor kamar |
| 4 | getTipeKamar() | functional | Mengembalikan nilai tipe kamar |
| 5 | getKapasitasMaksimal() | functional | Mengembalikan nilai kapasitas maksimal kamar |
| 6 | getHargaPerMalam() | functional | Mengembalikan nilai harga per malam |
| 7 | getIsTersedia() | functional | Mengembalikan status ketersediaan kamar (true/false) |
| 8 | setTipeKamar(String tipeKamar) | procedural | Mengubah tipe kamar dengan validasi (hanya Reguler/Premium/Suite), jika tidak valid diubah ke Reguler |
| 9 | setHargaPerMalam(double hargaPerMalam) | procedural | Mengubah harga per malam dengan validasi minimal Rp50.000, jika kurang diubah ke 50000 |

| | | | |
|---|---|---|---|
| 10 | pesanKamar() | procedural | Memesan kamar tanpa parameter jumlah tamu, mengubah status isTersedia menjadi false |
| 11 | pesanKamar(int jumlahTamu) | procedural | Memesan kamar dengan validasi jumlah tamu tidak melebihi kapasitas, mengubah status isTersedia |
| 12 | batalPesan() | procedural | Membatalkan pemesanan kamar, mengubah status isTersedia menjadi true |
| 13 | hitungTotalBayar(int jumlahMalam) | functional | Menghitung total tagihan berdasarkan jumlah malam menginap |
| 14 | hitungTotalBayar(int jumlahMalam, String kodeVoucher) | functional | Menghitung total tagihan dengan diskon 20% jika kode PROMO dan minimal 3 malam |
| 15 | tampilkanInfoKamar() | procedural | Menampilkan semua informasi kamar (nomor, tipe, kapasitas, harga, status) ke layar |

## IV. Dokumentasi dan Pembahasan Code

```java
public class Main {
    public static void main(String[] args) {

        // Membuat objek KamarHotel
        KamarHotel kamar1 = new KamarHotel("101", "Reguler", 2);
        KamarHotel kamar2 = new KamarHotel("201", "Suite", 2, 800000);

        // Testing 1
        System.out.println("\nTesting 1");
        kamar1.setTipeKamar("Presidential");
        kamar1.setHargaPerMalam(-10000);

        // Testing 2 overloading
        System.out.println("\nTesting 2 Overloading");
        kamar2.pesanKamar(4);
        kamar2.pesanKamar(2);
        kamar2.pesanKamar();

        // Testing 3 Voucher
        System.out.println("\nTesting 3 Voucher");
        double tagihanKamar1 = kamar1.hitungTotalBayar(2, "PROMO");
        System.out.println("Total Tagihan Kamar 1: Rp" + tagihanKamar1);

        System.out.println();
        double tagihanKamar2 = kamar2.hitungTotalBayar(4, "PROMO");
        System.out.println("Total Tagihan Kamar 2: Rp" + tagihanKamar2);

        // Cetak struk
        System.out.println("\nSTRUK PEMESANAN KAMAR");
        kamar1.tampilkanInfoKamar();
        kamar2.tampilkanInfoKamar();
    }
}
```

Program ini dibuat untuk mensimulasikan sistem manajemen reservasi hotel yang mengintegrasikan prinsip enkapsulasi dan validasi logika bisnis secara otomatis. Program diawali dengan pendefinisian public class Main yang berfungsi sebagai kelas utama untuk menjalankan seluruh alur simulasi. Di dalam method utama main(), program melakukan instansiasi dua buah objek dari kelas KamarHotel menggunakan dua jenis konstruktor yang berbeda guna mendemonstrasikan fleksibilitas pembuatan objek. Objek kamar1 dibuat menggunakan konstruktor pendaftaran kilat dengan parameter nomor kamar "101", tipe "Reguler", dan kapasitas 2 orang. Sementara itu, objek kamar2 dibuat menggunakan konstruktor lengkap yang mencakup parameter nomor kamar "201", tipe "Suite", kapasitas 2 orang, serta harga per malam sebesar Rp 800.000.

Program kemudian melakukan serangkaian pengujian ketahanan sistem (testing) untuk membuktikan bahwa atribut objek terlindungi dari kesalahan operasional. Pada tahap pertama, program menguji metode setter pada kamar1 dengan memasukkan tipe kamar "Presidential" dan harga negatif -10000. Sesuai dengan logika enkapsulasi yang diterapkan, sistem akan secara otomatis menolak input tersebut dan memaksa nilai atribut kembali ke standar minimum, yaitu tipe "Reguler" dan harga 50000. Selanjutnya, program mendemonstrasikan konsep method overloading melalui pemanggilan method pesanKamar(). Program

mensimulasikan kegagalan pemesanan pada kamar2 ketika jumlah tamu yang dimasukkan adalah 4 orang karena melebihi kapasitas maksimal, serta menunjukkan peringatan sistem saat mencoba memesan kamar yang statusnya sudah terisi.

Pada bagian akhir, program menjalankan simulasi perhitungan tagihan menggunakan method hitungTotalBayar() yang juga menerapkan overloading untuk memproses penggunaan kode voucher. Program menunjukkan validasi sistem di mana kamar1 gagal mendapatkan diskon karena durasi menginap kurang dari 3 malam meskipun menggunakan kode "PROMO", sedangkan kamar2 berhasil mendapatkan potongan harga sebesar 20% karena memenuhi seluruh syarat kriteria. Seluruh rangkaian simulasi ditutup dengan memanggil method tampilkanInfoKamar() pada kedua objek untuk mencetak "Struk Informasi" secara rapi ke layar, yang menampilkan kondisi akhir dari seluruh atribut kamar setelah melalui berbagai proses validasi dan transaksi.

```java
public class KamarHotel {
    // Mengunci Atribut Kamar Hotel
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;

    // Constructor Pertama Untuk Pendaftaran Kilat
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0.0;
        this.isTersedia = true;
    }

    // Constructor Kedua Untuk Pendaftaran Lengkap
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double
hargaPerMalam) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(hargaPerMalam);
        this.isTersedia = true;
    }
```

Setelah itu, masuk pada bagian KamarHotel.java. Program pada bagian ini mendefinisikan struktur inti dari sistem reservasi melalui kelas KamarHotel dengan menerapkan prinsip Encapsulation secara ketat. Pada bagian awal kelas, program mendeklarasikan lima atribut utama sebagai identitas kamar, yaitu nomorKamar, tipeKamar, kapasitasMaksimal, hargaPerMalam, dan isTersedia. Seluruh atribut ini diberikan modifier akses private untuk mengunci data sehingga tidak dapat dimodifikasi secara langsung dari luar kelas, yang bertujuan untuk menjaga integritas data sesuai dengan kaidah Information Hiding.

Untuk memfasilitasi pembuatan objek, kelas ini menyediakan dua jenis konstruktor melalui konsep Overloading Konstruktor. Konstruktor pertama

dirancang untuk kebutuhan "Pendaftaran Kilat", di mana sistem hanya menerima parameter nomor, tipe, dan kapasitas. Di dalam konstruktor ini, program secara otomatis menginisialisasi nilai hargaPerMalam menjadi 0.0 dan mengatur status isTersedia menjadi true secara bawaan. Uniknya, pada penetapan tipe kamar, konstruktor ini memanggil method setTipeKamar(tipeKamar) alih-alih melakukan penugasan langsung, guna memastikan nilai yang masuk tetap melewati filter validasi bisnis sejak objek pertama kali diciptakan.

Konstruktor kedua melayani "Pendaftaran Lengkap" dengan menerima empat parameter utama, termasuk harga per malam. Serupa dengan konstruktor sebelumnya, proses inisialisasi pada konstruktor ini juga memanfaatkan pemanggilan method setter seperti setTipeKamar dan setHargaPerMalam. Hal ini memastikan bahwa aturan bisnis seperti standar tipe kamar dan batas minimum harga, langsung diterapkan saat objek dikonstruksi di memori. Penggunaan keyword this pada kedua konstruktor berfungsi untuk membedakan antara variabel instansi kelas dengan parameter yang diterima dari luar, sehingga proses pemetaan data ke dalam atribut objek berjalan dengan akurat.

```java
public String getNomorKamar() {
        return this.nomorKamar; }

    public String getTipeKamar() {
        return this.tipeKamar; }

    public int getKapasitasMaksimal() {
        return this.kapasitasMaksimal; }

    public double getHargaPerMalam() {
        return this.hargaPerMalam; }

    public boolean getIsTersedia() {
        return this.isTersedia; }
```

Setelah pendefinisian atribut privat dan konstruktor selesai dilakukan, program mengimplementasikan serangkaian metode Getter sebagai jalur resmi untuk mengakses data kamar secara aman tanpa melanggar prinsip Encapsulation. Pada metode getNomorKamar(), program menggunakan kata kunci return untuk mengembalikan nilai string yang tersimpan dalam atribut nomorKamar sehingga kelas luar seperti Main dapat menampilkan identitas fisik kamar pada struk pemesanan. Logika yang serupa juga diterapkan pada metode getTipeKamar(), di mana program mengembalikan status kategori kamar yang sebelumnya telah melewati proses validasi ketat, guna memastikan informasi kelas layanan yang diterima oleh sistem penagihan adalah akurat dan sesuai standar perusahaan.

Selanjutnya, program menyediakan metode getKapasitasMaksimal() yang mengembalikan nilai integer untuk digunakan oleh sistem dalam melakukan validasi jumlah tamu sebelum proses pemesanan dinyatakan berhasil, guna mencegah terjadinya pelanggaran batas muatan kamar. Untuk keperluan kalkulasi finansial, program mengimplementasikan metode getHargaPerMalam() yang

memberikan akses baca terhadap nominal biaya sewa per malam, yang nantinya akan diproses lebih lanjut oleh metode hitung total bayar jika terdapat penggunaan voucher promo. Terakhir, program memiliki metode getIsTersedia() yang mengembalikan nilai boolean sebagai indikator utama bagi sistem untuk menentukan apakah suatu kamar masih dapat dipesan atau harus memberikan sinyal penolakan karena sedang ditempati oleh tamu lain

```java
// Setter yang diperlukan
    public void setTipeKamar(String tipeKamar) { // Set Tipe Kamar
        if (tipeKamar.equalsIgnoreCase("Reguler") || tipeKamar.equalsIgnoreCase("Premium") ||
tipeKamar.equalsIgnoreCase("Suite")) {
            this.tipeKamar = tipeKamar;
        } else {
            System.out.println("Tipe kamar tidak valid, akan diubah menjadi 'Reguler'.");
            this.tipeKamar = "Reguler";
        }
    }

    public void setHargaPerMalam(double hargaPerMalam) { // Set Harga Per Malam
        if (hargaPerMalam >= 50000) {
            this.hargaPerMalam = hargaPerMalam;
        } else {
            System.out.println("
Harga per malam kurang dari Rp50.000, akan diubah menjadi Rp50.000.");
            this.hargaPerMalam = 50000;
        }
    }

    // Metode Pemesanan
    public void pesanKamar() {
        if (this.isTersedia) {
            this.isTersedia = false;
            System.out.println("Kamar " + this.nomorKamar + " berhasil dipesan");
        } else {
            System.out.println("Kamar " + this.nomorKamar + " sudah terisi");
        }
    }
```

Setelah pendefinisian metode Getter, program mengimplementasikan metode Setter sebagai mekanisme kendali utama untuk memastikan integritas data melalui validasi bisnis yang ketat sebelum melakukan perubahan pada atribut privat. Pada metode setTipeKamar(), program menggunakan struktur percabangan if-else yang memanfaatkan fungsi equalsIgnoreCase untuk memverifikasi apakah input pengguna sesuai dengan kategori "Reguler", "Premium", atau "Suite". Jika input tidak memenuhi salah satu dari ketiga kriteria tersebut, program akan menjalankan blok else yang bertugas mencetak pesan peringatan tentang ketidakvalidan tipe kamar dan secara otomatis memaksa nilai atribut tipeKamar menjadi "Reguler" sebagai standar pengaman sistem.

Logika validasi serupa juga diterapkan pada metode setHargaPerMalam(), di mana program melakukan pengecekan apakah nominal harga yang dimasukkan bernilai minimal 50000. Apabila harga yang diinput berada di bawah ambang batas tersebut, sistem secara cerdas akan memberikan notifikasi koreksi kepada pengguna dan langsung menetapkan nilai atribut hargaPerMalam menjadi angka 50000 demi menjaga konsistensi kebijakan harga hotel. Selain metode Setter,

bagian ini juga memperkenalkan metode perilaku (behavior) pertama yaitu pesanKamar() tanpa parameter yang berfungsi untuk mengubah status ketersediaan objek secara langsung. Di dalam metode tersebut, program melakukan validasi terhadap atribut isTersedia; jika bernilai true, maka sistem akan mengubahnya menjadi false serta mencetak pesan keberhasilan pemesanan berdasarkan nomor kamar terkait, namun jika kamar sudah terisi, program akan memberikan pesan penolakan yang menyatakan bahwa kamar tersebut tidak dapat dipesan kembali.

```java
    public void pesanKamar(int jumlahTamu) { // Metode Pemesanan Overloading
        if (!this.isTersedia) {
            System.out.println("Kamar " + this.nomorKamar + " sudah terisi");
        } else if (jumlahTamu > this.kapasitasMaksimal) {
            System.out.println("Pemesanan belum berhasil, jumlah tamu melebihi kapasitas kamar (" + this.kapasitasMaksimal + "
orang)");
        } else {
            this.isTersedia = false;
            System.out.println("Kamar " + this.nomorKamar + " berhasil dipesan untuk " + jumlahTamu + " tamu");
        }
    }

    public void batalPesan() { // Metode Batal Pemesanan
        this.isTersedia = true;
        System.out.println("Kamar " + this.nomorKamar + " telah dibatalkan");
    }

    // Metode Pembayaran
    public double hitungTotalBayar(int jumlahMalam) {
        return jumlahMalam * this.hargaPerMalam;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) { // Metode Pembayaran Overloading
        double totalTagihan = hitungTotalBayar(jumlahMalam);

        if (kodeVoucher.equals("PROMO") && jumlahMalam >= 3) {
            System.out.println("Voucher berhasil digunakan, mendapatkan diskon 20%");
            return totalTagihan * 0.8;
        } else {
            System.out.println("Kode invalid atau syarat minimal 3 malam tidak terpenuhi");
            return totalTagihan;
        }
    }

    // Menampilkan Informasi Kamar
    public void tampilkanInfoKamar() {
        System.out.println("Nomor Kamar : " + this.nomorKamar);
        System.out.println("Tipe Kamar  : " + this.tipeKamar);
        System.out.println("Kapasitas   : " + this.kapasitasMaksimal + " Orang");
        System.out.println("Harga/Malam : Rp" + this.hargaPerMalam);
        System.out.println("Status      : " + (this.isTersedia ? "Tersedia" : "Terisi"));
        System.out.println("----------------------------------");
    }
}
```

Setelah mengimplementasikan metode pemesanan dasar, program memperluas fungsionalitas sistem melalui konsep Method Overloading dengan menghadirkan metode pesanKamar(int jumlahTamu) yang dilengkapi logika validasi lebih kompleks. Di dalam metode ini, program pertama-tama memeriksa status ketersediaan kamar, di mana jika kamar sudah terisi, sistem akan memberikan notifikasi penolakan. Namun, jika kamar tersedia, program akan melanjutkan ke pengecekan variabel jumlahTamu terhadap atribut kapasitasMaksimal; apabila jumlah tamu melebihi batas muatan, sistem secara otomatis memblokir pesanan dengan memberikan pesan teguran khusus dan menjaga status kamar tetap true (tersedia), sedangkan jika kapasitas mencukupi, program akan mengubah status kamar menjadi false dan mencetak konfirmasi keberhasilan. Selain itu, tersedia pula metode batalPesan() yang secara sederhana mengembalikan nilai atribut isTersedia menjadi true agar kamar dapat dipesan kembali oleh pelanggan lain.

Program kemudian mengimplementasikan sistem penagihan yang juga mengandalkan teknik Method Overloading melalui metode hitungTotalBayar() guna memberikan fleksibilitas dalam perhitungan biaya. Metode pertama yang hanya menerima parameter jumlahMalam berfungsi mengembalikan hasil perkalian durasi menginap dengan harga standar per malam. Sementara itu, metode kedua dengan tambahan parameter kodeVoucher memiliki logika bisnis yang lebih spesifik, di mana program akan memvalidasi apakah kode yang dimasukkan adalah "PROMO" dan apakah tamu menginap minimal selama 3 malam. Jika kedua syarat tersebut terpenuhi, sistem akan memberikan potongan harga sebesar 20%, namun jika salah satu syarat gagal terpenuhi, program akan mengembalikan nilai tagihan normal disertai pesan penolakan penggunaan voucher. Seluruh struktur kelas ini ditutup dengan metode tampilkanInfoKamar() yang bertugas mencetak rangkuman profil kamar secara rapi ke layar, mencakup seluruh detail atribut mulai dari nomor, tipe, kapasitas, harga, hingga status ketersediaan yang diformat menggunakan operator ternary untuk memudahkan pembacaan oleh pengguna.

## V. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan konsep inti Object-Oriented Programming (OOP) seperti Class, Object, Method, Constructor, dan prinsip Encapsulation dalam pemrograman Java dapat digunakan untuk membangun arsitektur sistem yang jauh lebih terstruktur, modular, dan aman. Dalam program reservasi hotel yang dibuat, seluruh atribut krusial seperti tipe kamar dan harga sewa dilindungi menggunakan modifier akses private untuk menerapkan kaidah Information Hiding, di mana perubahan data hanya dapat dilakukan melalui metode Setter yang telah terintegrasi dengan filter validasi bisnis untuk mencegah kesalahan input operasional.

Program ini juga menunjukkan bagaimana penggunaan Method Overloading pada konstruktor dan metode fungsional memberikan fleksibilitas tinggi dalam menangani berbagai skenario, mulai dari perbedaan jenis pendaftaran kamar hingga sistem penagihan yang mampu memproses kode voucher secara dinamis. Selain itu, penerapan logika validasi di dalam metode perilaku (behavior) seperti pengecekan kapasitas maksimal tamu terbukti efektif dalam memetakan aturan dunia nyata ke dalam sistem digital guna menjaga konsistensi data. Hasil dari seluruh proses transaksi tersebut, mulai dari pengujian ketahanan setter hingga kalkulasi diskon otomatis, ditampilkan dalam bentuk "Struk Informasi" yang memberikan ringkasan lengkap mengenai kondisi akhir atribut objek. Dengan demikian, praktikum ini memberikan pemahaman mendalam mengenai peran penting enkapsulasi dan manajemen objek dalam menyelesaikan studi kasus yang melibatkan logika bisnis kompleks dan proteksi integritas data secara sistematis.

## VI. Daftar Pustaka

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemrograman Berorientasi Objek Modul 4: The Core of OOP*. Program Studi Ilmu Komputer, Fakultas Sains dan Komputer, Universitas Pertamina.

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemrograman Berorientasi Objek Modul 5: Encapsulation*. Program Studi Ilmu Komputer, Fakultas Sains dan Komputer, Universitas Pertamina.